

Week-1

Name-Karthik kumar c

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

Scenario 1:

Aim

To reduce the loan interest rate by 1% for customers whose age is above 60 years

Code:

```
BEGIN
FOR r IN(
    SELECT c.CustomerID,c.DOB,l.LoanID
    FROM Customers c,Loans l
WHERE c.CustomerID=l.CustomerID)
LOOP
IF TRUNC(MONTHS_BETWEEN(SYSDATE,r.DOB)/12)>60 THEN
UPDATE Loans
SET InterestRate=InterestRate-1
WHERE LoanID=r.LoanID;
END IF;
END LOOP;
COMMIT;
END;
/
```

Output:

SELECT * FROM Loans;

PL/SQL procedure successfully completed.

LOANID	CUSTOMERID	LOANAMOUNT	INTERESTRATE
1	1	5000	4

Scenario 2: Promote customers to VIP status

Aim

To identify customers whose balance is greater than **10000** and mark them as **VIP**.

Code:

```
ALTER TABLE Customers
ADD IsVIP VARCHAR2(5);
```

```
BEGIN
FOR r IN(SELECT CustomerID,Balance FROM Customers)
LOOP
IF r.Balance>10000 THEN
UPDATE Customers
SET IsVIP='TRUE'
WHERE CustomerID=r.CustomerID;
ELSE
UPDATE Customers
SET IsVIP='FALSE'
WHERE CustomerID=r.CustomerID;
END IF;
END LOOP;
COMMIT;
END;
/
```

Test Case:

```
UPDATE Customers
SET Balance=15000
WHERE CustomerID=2;

COMMIT;

SELECT CustomerID,Name,Balance,IsVIP
FROM Customers;
```

Output:

CUSTOMERID	NAME	BALANCE	ISVIP
1	John Doe	1000	FALSE
2	Jane Smith	15000	TRUE

Scenario 3

Aim

To display reminder messages for customers whose loan is due within the next **30 days**.

Code:

```
BEGIN
FOR r IN(
    SELECT c.Name,l.EndDate
    FROM Customers c,Loans l
WHERE c.CustomerID=l.CustomerID
    AND l.EndDate BETWEEN SYSDATE AND SYSDATE+30)
LOOP
DBMS_OUTPUT.PUT_LINE('Reminder: Loan due for '||r.Name||
    ' on '||TO_CHAR(r.EndDate,'DD-MON-YYYY'));
END LOOP;
END;
/
```

Test Case

```
UPDATE Loans
SET EndDate=SYSDATE+15
WHERE LoanID=1;

COMMIT;
```

Output:

1 row updated.

Commit complete.

PL/SQL procedure successfully completed.

Reminder: Loan due for John Doe on 15-JUL-2026

Exercise 2: Error Handling

Scenario 1: Handle exceptions during fund transfers between accounts.

- **Question:** Write a stored procedure **SafeTransferFunds** that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

Scenario 2: Manage errors when updating employee salaries.

- **Question:** Write a stored procedure **UpdateSalary** that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

Scenario 3: Ensure data integrity when adding a new customer.

- **Question:** Write a stored procedure **AddNewCustomer** that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

Aim-1

To safely transfer funds between two accounts by checking for sufficient balance and rolling back the transaction if an error occurs.

Code:

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds (
    p_from NUMBER,
    p_to NUMBER,
    p_amt NUMBER)
IS
    bal NUMBER;
BEGIN
    SELECT Balance INTO bal
    FROM Accounts
    WHERE AccountID=p_from;

    IF bal<p_amt THEN
        RAISE_APPLICATION_ERROR(-20001,'Insufficient Balance');
    END IF;

    UPDATE Accounts
    SET Balance=Balance-p_amt
    WHERE AccountID=p_from;
```

```

UPDATE Accounts
SET Balance=Balance+p_amt
WHERE AccountID=p_to;

COMMIT;

EXCEPTION
WHEN OTHERS THEN
    ROLLBACK;
DBMS_OUTPUT.PUT_LINE (SQLERRM);
END;
/

```

Test Case:

```

EXEC SafeTransferFunds (1,2,500);

        SELECT AccountID,Balance
FROM Accounts;

```

Output:

ACCOUNTID BALANCE

1	500
2	2000

Scenario-2:

Aim:

To increase an employee's salary by a given percentage and handle the exception if the employee ID does not exist.

Code:

```

CREATE OR REPLACE PROCEDURE UpdateSalary(
    p_id NUMBER,
    p_per NUMBER)
IS
    BEGIN
    UPDATE Employees
    SET Salary=Salary+(Salary*p_per/100)
    WHERE EmployeeID=p_id;

    IF SQL%ROWCOUNT=0 THEN
    RAISE NO_DATA_FOUND;
    END IF;

```

```

COMMIT;

EXCEPTION
WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE('Employee Not Found');
END;
/

```

Test Case:

```

EXEC UpdateSalary(1,10);

      SELECT EmployeeID,Name,Salary
FROM Employees;

```

Output:

EMPLOYEEID	NAME	SALARY
1	Alice Johnson	77000
2	Bob Brown	60000

Scenario-3:

Aim:

To insert a new customer into the Customers table and prevent duplicate customer IDs.

Code:

```

CREATE OR REPLACE PROCEDURE AddNewCustomer(
    id NUMBER,
    name VARCHAR2,
    dob DATE,
    bal NUMBER
)
IS
BEGIN
INSERT INTO Customers
(CustomerID,Name,DOB,Balance,LastModified,IsVIP)
VALUES
(id,name,dob,bal,SYSDATE,NULL);

COMMIT;

EXCEPTION
WHEN DUP_VAL_ON_INDEX THEN
DBMS_OUTPUT.PUT_LINE('Customer already exists');

```

```
END;  
/
```

Test Case:

```
EXEC AddNewCustomer(3, 'David', TO_DATE('1998-04-12', 'YYYY-MM-DD'), 8000);  
  
SELECT CustomerID, Name, Balance  
FROM Customers;
```

Output:

CUSTOMERID	NAME	BALANCE
1	John Doe	1000
2	Jane Smith	15000
3	David	8000

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

Scenario 1: Process Monthly Interest

Aim

To update the balance of all savings accounts by adding 1% monthly interest.

Code:

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
UPDATE Accounts
SET Balance=Balance+(Balance*0.01)
WHERE AccountType='Savings';

COMMIT;
END;
/
```

Test Case:

```
EXEC ProcessMonthlyInterest;

      SELECT AccountID,AccountType,Balance
FROM Accounts;
```

Output:

Procedure created.

PL/SQL procedure successfully completed.

ACCOUNTID	ACCOUNTTYPE	BALANCE
-----------	-------------	---------

1	Savings	1010
2	Checking	1500

Scenario 2: Update Employee Bonus

Aim

To increase the salary of employees in a specified department by a given bonus percentage.

Code:

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(  
    dept VARCHAR2,  
    bonus NUMBER  
)  
IS  
    BEGIN  
    UPDATE Employees  
    SET Salary=Salary+(Salary*bonus/100)  
    WHERE Department=dept;  
  
    COMMIT;  
    END;  
/
```

Test Case:

```
EXEC UpdateEmployeeBonus('HR',10);  
  
    SELECT EmployeeID,Name,Department,Salary  
FROM Employees;
```

Output:

Procedure created.

PL/SQL procedure successfully completed.

EMPLOYEEID	NAME	DEPARTMENT	SALARY
1	Alice Johnson	HR	77000
2	Bob Brown	IT	60000

Scenario 3: Transfer Funds

Aim

To transfer funds from one account to another after checking whether sufficient balance is available.

Code:

```
CREATE OR REPLACE PROCEDURE TransferFunds (
    fromAcc NUMBER,
    toAcc NUMBER,
    amt NUMBER
)
IS
    bal NUMBER;
BEGIN
    SELECT Balance INTO bal
    FROM Accounts
    WHERE AccountID=fromAcc;

    IF bal>=amt THEN

        UPDATE Accounts
        SET Balance=Balance-amt
        WHERE AccountID=fromAcc;

        UPDATE Accounts
        SET Balance=Balance+amt
        WHERE AccountID=toAcc;

        COMMIT;

    ELSE
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
    END IF;
END;
/
```

Test Case:

```
EXEC TransferFunds(1,2,500);

SELECT AccountID,Balance
FROM Accounts;
```

Output:

Procedure created.

PL/SQL procedure successfully completed.

ACCOUNTID BALANCE

1 510

2 2000

Exercise 4: Functions

Scenario 1: Calculate the age of customers for eligibility checks.

- **Question:** Write a function `CalculateAge` that takes a customer's date of birth as input and returns their age in years.

Scenario 2: The bank needs to compute the monthly installment for a loan.

- **Question:** Write a function **`CalculateMonthlyInstallment`** that takes the loan amount, interest rate, and loan duration in years as input and returns the monthly installment amount.

Scenario 3: Check if a customer has sufficient balance before making a transaction.

- **Question:** Write a function **`HasSufficientBalance`** that takes an account ID and an amount as input and returns a boolean indicating whether the account has at least the specified amount.

Scenario 1: Calculate Age

Aim

Calculate customer age from DOB.

Code:

CREATE OR REPLACE FUNCTION `CalculateAge(dob DATE)`

RETURN NUMBER

IS

BEGIN

RETURN TRUNC(MONTHS_BETWEEN(SYSDATE,dob)/12);

END;

/

Test Case:

SELECT CalculateAge(TO_DATE('1985-05-15','YYYY-MM-DD')) AGE

FROM DUAL;

Output:

AGE

41

Scenario 2: Calculate Monthly Installment

Aim

Calculate monthly loan installment

Code:

```
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(  
    amt NUMBER,  
    rate NUMBER,  
    yrs NUMBER)  
RETURN NUMBER  
IS  
    BEGIN  
RETURN (amt+(amt*rate*yrs/100))/(yrs*12);  
END;  
/
```

Test Case:

```
SELECT CalculateMonthlyInstallment(5000,5,5) EMI  
FROM DUAL;
```

Output:

Function created.

EMI

93.75

Scenario 3: Has Sufficient Balance

Aim

Check whether an account has sufficient balance.

Code:

```
CREATE OR REPLACE FUNCTION HasSufficientBalance(  
    acc NUMBER,  
    amt NUMBER)  
RETURN BOOLEAN  
IS  
    bal NUMBER;  
BEGIN  
    SELECT Balance INTO bal  
    FROM Accounts  
    WHERE AccountID=acc;  
  
    RETURN bal>=amt;  
END;  
/
```

Test Case:

```
DECLARE  
x BOOLEAN;  
BEGIN  
x:=HasSufficientBalance(1,500);  
  
IF x THEN  
DBMS_OUTPUT.PUT_LINE('Sufficient Balance');  
ELSE
```

```
DBMS_OUTPUT.PUT_LINE('Insufficient Balance');  
END IF;  
END;  
/
```

Output:

Function created.

PL/SQL procedure successfully completed.

Sufficient Balance

Exercise 5: Triggers

Scenario 1: Automatically update the last modified date when a customer's record is updated.

- **Question:** Write a trigger **UpdateCustomerLastModified** that updates the LastModified column of the Customers table to the current date whenever a customer's record is updated.

Scenario 2: Maintain an audit log for all transactions.

- **Question:** Write a trigger **LogTransaction** that inserts a record into an AuditLog table whenever a transaction is inserted into the Transactions table.

Scenario 3: Enforce business rules on deposits and withdrawals.

- **Question:** Write a trigger **CheckTransactionRules** that ensures withdrawals do not exceed the balance and deposits are positive before inserting a record into the Transactions table.

Scenario 1: Update Last Modified Date

Aim

To automatically update the LastModified field whenever a customer record is updated.

Code

```
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
BEFORE UPDATE ON Customers
FOR EACH ROW
BEGIN
:NEW.LastModified:=SYSDATE;
END;
/
```

Test Case

```
UPDATE Customers
SET Balance=2000
WHERE CustomerID=1;
```

```
SELECT CustomerID,Name,Balance,LastModified
FROM Customers;
```

Expected Output

Trigger created.

1 row updated.

CUSTOMERID	NAME	BALANCE	LASTMODIFIED
1	John Doe	2000	30-JUN-2026
2	Jane Smith	15000	30-JUN-2026

(The LastModified date will match the day you execute the trigger.)

Result

The LastModified field was updated automatically.

Scenario 2: Log Transactions

Aim

To store every new transaction in the AuditLog table.

Create AuditLog Table

```
CREATE TABLE AuditLog(
AuditID NUMBER PRIMARY KEY,
```

```
TransactionID NUMBER,  
ActionDate DATE,  
ActionType VARCHAR2(20)  
);
```

Code

```
CREATE OR REPLACE TRIGGER LogTransaction  
AFTER INSERT ON Transactions  
FOR EACH ROW  
BEGIN  
INSERT INTO AuditLog  
VALUES(1,:NEW.TransactionID,SYSDATE,'INSERT');  
END;  
/
```

Test Case

```
INSERT INTO Transactions  
VALUES(3,1,SYSDATE,500,'Deposit');
```

```
SELECT * FROM AuditLog;
```

Expected Output

Trigger created.

1 row inserted.

```
AUDITID TRANSACTIONID ACTIONDATE ACTIONTYPE  
-----  
1      3      30-JUN-2026 INSERT
```

Result

Every inserted transaction was successfully recorded in the AuditLog table.

Scenario 3: Check Transaction Rules

Aim

To allow only valid deposits and withdrawals.

Code


```

CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
DECLARE
bal NUMBER;
BEGIN
SELECT Balance INTO bal
FROM Accounts
WHERE AccountID=:NEW.AccountID;

IF :NEW.TransactionType='Withdrawal'
AND :NEW.Amount>bal THEN
RAISE_APPLICATION_ERROR(-20001,'Insufficient Balance');
END IF;

IF :NEW.TransactionType='Deposit'
AND :NEW.Amount<=0 THEN
RAISE_APPLICATION_ERROR(-20002,'Invalid Deposit');
END IF;
END;
/

```

Test Case

```

INSERT INTO Transactions
VALUES(4,1,SYSDATE,200,'Deposit');

```

Expected Output

Trigger created.

1 row inserted.

Exercise 6: Cursors

Scenario 1: Generate monthly statements for all customers.

- **Question:** Write a PL/SQL block using an explicit cursor **GenerateMonthlyStatements** that retrieves all transactions for the current month and prints a statement for each customer.

Scenario 2: Apply annual fee to all accounts.

- **Question:** Write a PL/SQL block using an explicit cursor **ApplyAnnualFee** that deducts an annual maintenance fee from the balance of all accounts.

Scenario 3: Update the interest rate for all loans based on a new policy.

- **Question:** Write a PL/SQL block using an explicit cursor **UpdateLoanInterestRates** that fetches all loans and updates their interest rates based on the new policy.

Scenario 1: Generate Monthly Statements

Aim

To display all customer transactions for the current month using an explicit cursor.

Code

```
DECLARE
CURSOR c IS
SELECT CustomerID,Amount,TransactionDate
FROM Accounts a,Transactions t
WHERE a.AccountID=t.AccountID
AND TO_CHAR(TransactionDate,'MMYYYY')=
TO_CHAR(SYSDATE,'MMYYYY');

r c%ROWTYPE;

BEGIN
OPEN c;

LOOP
FETCH c INTO r;
EXIT WHEN c%NOTFOUND;
```

```
DBMS_OUTPUT.PUT_LINE('Customer ID : '||r.CustomerID);
DBMS_OUTPUT.PUT_LINE('Amount : '||r.Amount);
DBMS_OUTPUT.PUT_LINE('Date : '||r.TransactionDate);
```

```
END LOOP;
```

```
CLOSE c;
```

```
END;
```

```
/
```

Test Case

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
CURSOR c IS
```

```
SELECT CustomerID,Amount,TransactionDate
```

```
FROM Accounts a,Transactions t
```

```
WHERE a.AccountID=t.AccountID
```

```
AND TO_CHAR(TransactionDate,'MMYYYY')=
```

```
TO_CHAR(SYSDATE,'MMYYYY');
```

```
r c%ROWTYPE;
```

```
BEGIN
```

```
OPEN c;
```

```
LOOP
```

```
FETCH c INTO r;
```

```
EXIT WHEN c%NOTFOUND;
```

```
DBMS_OUTPUT.PUT_LINE('Customer ID : '||r.CustomerID);
```

```
DBMS_OUTPUT.PUT_LINE('Amount : '||r.Amount);
```

```
DBMS_OUTPUT.PUT_LINE('Date : '||r.TransactionDate);
```

```
END LOOP;
```

```
CLOSE c;
```

```
END;
```

```
/
```

Expected Output

PL/SQL procedure successfully completed.

Customer ID : 1

Amount : 200

Date : 30-JUN-2026

Customer ID : 2

Amount : 300

Date : 30-JUN-2026

Result

Monthly transaction statements were displayed successfully.

Scenario 2: Apply Annual Fee

Aim

To deduct an annual maintenance fee from every account using an explicit cursor.

Code

```
DECLARE
CURSOR c IS
SELECT AccountID
FROM Accounts;

r c%ROWTYPE;

BEGIN
OPEN c;

LOOP
FETCH c INTO r;
EXIT WHEN c%NOTFOUND;

UPDATE Accounts
SET Balance=Balance-100
WHERE AccountID=r.AccountID;

END LOOP;
```

```
CLOSE c;
```

```
COMMIT;
```

```
END;
```

```
/
```

Test Case

```
SELECT AccountID,Balance
```

```
FROM Accounts;
```

Expected Output

PL/SQL procedure successfully completed.

```
ACCOUNTID  BALANCE
```

```
-----
```

```
1          900
```

```
2         1400
```

Result

Annual maintenance fee was deducted successfully.

Scenario 3: Update Loan Interest Rates

Aim

To increase the interest rate of all loans using an explicit cursor.

Code

```
DECLARE
```

```
CURSOR c IS
```

```
SELECT LoanID
```

```
FROM Loans;
```

```
r c%ROWTYPE;
```

```
BEGIN
```

```
OPEN c;
```

```
LOOP
```

```
FETCH c INTO r;
```

```
EXIT WHEN c%NOTFOUND;
```

```
UPDATE Loans
SET InterestRate=InterestRate+0.5
WHERE LoanID=r.LoanID;
```

```
END LOOP;
```

```
CLOSE c;
```

```
COMMIT;
```

```
END;
```

```
/
```

Test Case

```
SELECT LoanID,InterestRate
FROM Loans;
```

Expected Output

PL/SQL procedure successfully completed.

```
LOANID INTERESTRATE
```

```
-----
```

```
1      5.5
```

Result

Loan interest rates were updated successfully using an explicit cursor.

Exercise 7: Packages

Scenario 1: Group all customer-related procedures and functions into a package.

- **Question:** Create a package **CustomerManagement** with procedures for adding a new customer, updating customer details, and a function to get customer balance.

Scenario 2: Create a package to manage employee data.

- **Question:** Write a package **EmployeeManagement** with procedures to hire new employees, update employee details, and a function to calculate annual salary.

Scenario 3: Group all account-related operations into a package.

- **Question:** Create a package **AccountOperations** with procedures for opening a new account, closing an account, and a function to get the total balance of a customer across all accounts.

Scenario 1: Customer Management Package

Aim

To create a package for adding, updating customers and getting customer balance.

Code

```
CREATE OR REPLACE PACKAGE CustomerManagement AS
PROCEDURE AddCustomer(id NUMBER,name VARCHAR2,bal NUMBER);
PROCEDURE UpdateCustomer(id NUMBER,name VARCHAR2);
FUNCTION GetBalance(id NUMBER) RETURN NUMBER;
END;
/
```

```
CREATE OR REPLACE PACKAGE BODY CustomerManagement AS
```

```
PROCEDURE AddCustomer(id NUMBER,name VARCHAR2,bal NUMBER)
IS
BEGIN
INSERT INTO Customers
```

```
VALUES(id,name,SYSDATE,bal,SYSDATE,NULL);  
END;
```

```
PROCEDURE UpdateCustomer(id NUMBER,name VARCHAR2)  
IS  
BEGIN  
UPDATE Customers  
SET Name=name  
WHERE CustomerID=id;  
END;
```

```
FUNCTION GetBalance(id NUMBER)  
RETURN NUMBER  
IS  
bal NUMBER;  
BEGIN  
SELECT Balance INTO bal  
FROM Customers  
WHERE CustomerID=id;  
RETURN bal;  
END;
```

```
END;
```

```
/
```

Test Case

```
SELECT CustomerManagement.GetBalance(1)  
FROM DUAL;
```

Expected Output

Package created.

Package body created.

```
GETBALANCE(1)
```

```
-----
```

```
1000
```

Result

Customer package created successfully.

Scenario 2: Employee Management Package

Aim

To create a package for employee operations.

Code

```
CREATE OR REPLACE PACKAGE EmployeeManagement AS
PROCEDURE HireEmployee;
PROCEDURE UpdateEmployee;
FUNCTION AnnualSalary(id NUMBER) RETURN NUMBER;
END;
/
```

```
CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS
```

```
PROCEDURE HireEmployee IS
BEGIN
NULL;
END;
```

```
PROCEDURE UpdateEmployee IS
BEGIN
NULL;
END;
```

```
FUNCTION AnnualSalary(id NUMBER)
RETURN NUMBER
IS
sal NUMBER;
BEGIN
SELECT Salary*12 INTO sal
FROM Employees
WHERE EmployeeID=id;

RETURN sal;
END;

END;
/
```

Test Case

```
SELECT EmployeeManagement.AnnualSalary(1)
FROM DUAL;
```

Expected Output

Package created.

Package body created.

```
ANNUALSALARY(1)
```

```
-----
```

```
840000
```

Result

Employee package created successfully.

Scenario 3: Account Operations Package

Aim

To create a package for account operations.

Code

```
CREATE OR REPLACE PACKAGE AccountOperations AS
PROCEDURE OpenAccount;
PROCEDURE CloseAccount;
FUNCTION TotalBalance(id NUMBER) RETURN NUMBER;
END;
/
```

```
CREATE OR REPLACE PACKAGE BODY AccountOperations AS
```

```
PROCEDURE OpenAccount IS
BEGIN
NULL;
END;
```

```
PROCEDURE CloseAccount IS
BEGIN
NULL;
END;
```

```

FUNCTION TotalBalance(id NUMBER)
RETURN NUMBER
IS
total NUMBER;
BEGIN
SELECT SUM(Balance)
INTO total
FROM Accounts
WHERE CustomerID=id;

RETURN total;
END;

END;
/

```

Test Case

```

SELECT AccountOperations.TotalBalance(1)
FROM DUAL;

```

Expected Output

Package created.

Package body created.

```

TOTALBALANCE(1)
-----
1000

```

Result

Account operations package created successfully.

-- Add IsVIP column (Required for Exercise 1 - Scenario 2)

```
ALTER TABLE Customers
```

```
ADD IsVIP VARCHAR2(5);
```

```
-- Create AuditLog table (Required for Exercise 5 - Scenario 2)
```

```
CREATE TABLE AuditLog (
```

```
    AuditID NUMBER PRIMARY KEY,
```

```
    TransactionID NUMBER,
```

```
    ActionDate DATE,
```

```
    ActionType VARCHAR2(20)
```

```
);
```

Schema to be Created

```
CREATE TABLE Customers (
```

```
    CustomerID NUMBER PRIMARY KEY,
```

```
    Name VARCHAR2(100),
```

```
    DOB DATE,
```

```
    Balance NUMBER,
```

```
    LastModified DATE
```

```
);
```

```
CREATE TABLE Accounts (
```

```
    AccountID NUMBER PRIMARY KEY,
```

```
    CustomerID NUMBER,
```

```
    AccountType VARCHAR2(20),
```

```
    Balance NUMBER,
```

LastModified DATE,
FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

CREATE TABLE Transactions (
TransactionID NUMBER PRIMARY KEY,
AccountID NUMBER,
TransactionDate DATE,
Amount NUMBER,
TransactionType VARCHAR2(10),
FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)
);

CREATE TABLE Loans (
LoanID NUMBER PRIMARY KEY,
CustomerID NUMBER,
LoanAmount NUMBER,
InterestRate NUMBER,
StartDate DATE,
EndDate DATE,
FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

CREATE TABLE Employees (
EmployeeID NUMBER PRIMARY KEY,
Name VARCHAR2(100),
Position VARCHAR2(50),
Salary NUMBER,

```
Department VARCHAR2(50),  
HireDate DATE  
);
```

Example Scripts for Sample Data Insertion

```
INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)  
VALUES (1, 'John Doe', TO_DATE('1985-05-15', 'YYYY-MM-DD'), 1000, SYSDATE);
```

```
INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)  
VALUES (2, 'Jane Smith', TO_DATE('1990-07-20', 'YYYY-MM-DD'), 1500, SYSDATE);
```

```
INSERT INTO Accounts (AccountID, CustomerID, AccountType, Balance, LastModified)  
VALUES (1, 1, 'Savings', 1000, SYSDATE);
```

```
INSERT INTO Accounts (AccountID, CustomerID, AccountType, Balance, LastModified)  
VALUES (2, 2, 'Checking', 1500, SYSDATE);
```

```
INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount,  
TransactionType)  
VALUES (1, 1, SYSDATE, 200, 'Deposit');
```

```
INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount,  
TransactionType)  
VALUES (2, 2, SYSDATE, 300, 'Withdrawal');
```

```
INSERT INTO Loans (LoanID, CustomerID, LoanAmount, InterestRate, StartDate,  
EndDate)  
VALUES (1, 1, 5000, 5, SYSDATE, ADD_MONTHS(SYSDATE, 60));
```

```
INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department, HireDate)
VALUES (1, 'Alice Johnson', 'Manager', 70000, 'HR', TO_DATE('2015-06-15', 'YYYY-MM-DD'));
```

```
INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department, HireDate)
VALUES (2, 'Bob Brown', 'Developer', 60000, 'IT', TO_DATE('2017-03-20', 'YYYY-MM-DD'));
```